Next, we need to have Composer parse that file and follow its instructions. Because we love Docker (and because you might be on an older system with an older version of Composer, if at all), we'll run `composer` from a container. This will be much like we've been doing things with PHP, where we mount the current directory into the container so that, when the work is done, the results remain on the host disk.

```
$ alias dcomposer='docker run --rm --interactive --tty --volume $PWD:/app
composer:2.8'
$ dcomposer install
...
  - Installing twig/twig (4.x-dev c649c16): Extracting archive
  - Locking twig/html-extra (4.x-dev a1b1158)
```

This installs Twig core and its `HtmlExtension` (which is not installed by default for some reason). Some examples in this section require that `Twig\Extra\Html\HtmlExtension` are available. Now create an `index.php` file, where we'll pull in the composer-generated `autoload.php` file (this was created when `composer install` executed) and some boilerplate to get Twig ready:

```
// index.php
require_once __DIR__ . '/vendor/autoload.php';
$loader = new \Twig\Loader\FilesystemLoader(__DIR__ . '/views');
$twig = new \Twig\Environment($loader, [
    'cache' => __DIR__ . '/cache',
]);
```

The chunk of code above is important: it's the minimum boilerplate required to get `Twig\Environment` bootstrapped and ready for work. For the rest of this section, wherever a code example contains a `$twig` variable, assume the three lines above are present unless specified otherwise!

The heart of Twig is a `Twig\Environment` object, which stores configuration and loads templates via a loader – here we've used the built-in `FilesystemLoader`, which will search for template files on the path given to it at instantiation (you can configure multiple paths by instantiating with an array of paths or adding them with `FilesystemLoader::addPath()`). The `Environment` object constructor also accepts an associative array of config options – we've set the `cache` option here to configure where Twig will cache compiled templates. All that's left to do now is to create our first Twig template and render it.